CAFCL: Complexity-Aware Curriculum Learning for Code Generation

Qixin Chen
Faculty of Information Science and Engineering, Ocean University of China
Qingdao, China
chenqixin@stu.ouc.edu.cn

***Abstract*—Large language models (LLMs) have recently demonstrated tremendous potential in code generation. However, existing fine-tuning approaches suffer from two pervasive issues: first, they lack systematic control over training-data quality—low-quality samples (e.g., non-executable code, instruction-code mismatches) disturb model learning; second, they ignore the impact of how sample complexity is organized during training. To address these issues, this paper proposes a Complexity-Aware Filtering and Curriculum Learning (CAFCL) framework for code generation. The framework consists of two core components: (1) a complexity-aware multi-dimensional data-quality filtering strategy that systematically evaluates and filters evol-instruct data along three dimensions—instruction complexity, code executability, and instruction-code alignment; and (2) a complexity-graduated curriculum learning training strategy that partitions the filtered data into multiple difficulty buckets using the complexity scores and organizes training in an easy-to-hard order. We conduct systematic experiments on the Qwen3-0.6B model and evaluate on the HumanEval benchmark. Results show that CAFCL reaches a pass@1 of 32.3, improving over the unfine-tuned base model (28.05) by 4.27 percentage points and slightly outperforming full-data random fine-tuning (31.71) under an equal training budget. Training-dynamics analysis further reveals the stabilization effect and stage-wise convergence pattern induced by curriculum learning.**

Keywords—Data Quality Filtering; Supervised Fine-Tuning; Large Language Models; Code Generation; Curriculum Learning;

I. INTRODUCTION

Code generation—the automatic translation of natural-language requirements into executable programs—is a long-standing goal of artificial intelligence [2]. With the rise of large language models (LLMs), particularly Transformer-based models such as the GPT series [1], LLaMA [5], and Qwen [6], the field has witnessed unprecedented breakthroughs. By pre-training on massive code corpora such as The Stack [7], these models acquire rich knowledge of programming syntax and patterns, enabling them to perform tasks like code completion and simple function generation [2, 5].

The real challenge in code generation, however, lies in understanding and implementing instructions that involve complex logic, multiple constraints, and specific edge conditions. Unlike general text generation, code is judged against objective and strict criteria: it must pass unit tests and realize the intended functionality [2, 12]. Early code-generation models often fared poorly on such complex tasks—the generated code might contain logical errors, fail to handle edge cases, or entirely misunderstand user intent [5, 12]. Although recent work such as StarCoder [4] and Code Llama [5] has continued to scale up parameter counts and data volume, the ability to follow complex instructions remains limited.

Supervised fine-tuning (SFT) is an effective means of bridging pre-trained knowledge and task requirements [8, 9], and its effectiveness is highly sensitive to training-data quality [10]. The recently proposed Evol-Instruct method [10] (from WizardLM) uses automated iteration to generate highly complex instructions, significantly improving instruction-following ability. However, we observe that current evol-instruct-based code fine-tuning paradigms still have two key shortcomings:

First, data quality lacks systematic assurance. The automated generation process in Evol-Instruct inevitably introduces noisy samples, including syntactically broken (non-executable) code, "off-topic" samples where instructions and code do not match semantically, and degenerate samples that have lost their intended complexity. Prior work has shown that noise in training data significantly reduces learning efficiency and can even introduce harmful biases [10, 15]. Yet most existing work directly uses the raw evol-instruct data without a multi-dimensional, code-specific quality-filtering mechanism.

Second, the training process neglects sample-complexity organization. Conventional SFT randomly shuffles all training samples and trains uniformly, failing to exploit the difficulty gradient among samples. Both cognitive science and curriculum learning theory [16] indicate that a progressive easy-to-hard learning schedule helps learners build a more robust knowledge structure. In the code-generation setting, letting the model first master basic syntactic patterns and simple logic, then gradually transition to multi-constraint, deeply nested tasks, is expected to yield more efficient knowledge accumulation.

Motivated by these observations, we propose the Complexity-Aware Filtering and Curriculum Learning (CAFCL) framework, which consists of two synergistic components: (1) a complexity-aware multi-dimensional data-quality filtering strategy that systematically filters low-quality samples along three dimensions—instruction complexity, code executability, and instruction-code alignment; and (2) a complexity-graduated curriculum learning strategy that, using the complexity scores produced during filtering, partitions the data into difficulty buckets and guides the model through an easy-to-hard learning schedule. We conduct systematic experiments on Qwen3-0.6B

and validate the effectiveness of the framework on the HumanEval benchmark [2].

The main contributions of this paper are:

- We propose a multi-dimensional quality-filtering strategy tailored to evol-instruct code data that combines instruction complexity, executability verification, and instruction-code alignment, effectively removing noisy samples.
- We design a complexity-score-based curriculum learning scheme that improves fine-tuning efficiency and effectiveness through progressive difficulty organization.
- We perform a systematic empirical evaluation on HumanEval, including multi-baseline comparisons and training-dynamics analysis, validating the framework's effectiveness.

II. Related Work

A. Large Language Models for Code Generation

The development of code generation is deeply intertwined with the evolution of LLMs. Early models such as OpenAI's GPT-3 [1] demonstrated zero- and few-shot learning abilities, but their code-generation capability was part of a general-purpose language ability and was not specifically optimized. Subsequently, code-specialized models began to appear, notably OpenAI's Codex [2], which was fine-tuned on large amounts of publicly available code and powered GitHub Copilot. DeepMind's AlphaCode [3] achieved human-level performance in programming contests by introducing massive code generation and filtering mechanisms. These works highlight the importance of model scale and code-specific data. In recent years, the open-source community has also produced many excellent code models, including StarCoder [4], Code Llama [5], and the Qwen [6] series used in this study, providing a strong foundation for academic and industrial research.

B. Code Instruction Fine-Tuning Datasets

The success of SFT depends heavily on high-quality instruction datasets, and their evolution can be summarized as a three-stage progression of "scale → quality → complexity." The first stage focused on large-scale pre-training corpora, with researchers crawling complete repositories from platforms such as GitHub—e.g., bigcode/the-stack [7] and codeparrot/github-code. These data were not manually filtered and were used to let models absorb general syntax and structure, but they lack an explicit instruction format. In the second stage, the "Self-Instruct" technique [8] was proposed: starting from a small set of seed tasks, strong teacher models (e.g., GPT-3.5) are used to automatically expand into large numbers of "instruction-code" pairs. CodeAlpaca [9] is a representative work of this stage, being the first to introduce self-instruct into the code domain; however, the generated instructions still lack diversity and complexity. The third stage emphasizes the evolution of instruction complexity: Evol-Instruct [10] iteratively imposes additional constraints, increases difficulty, or constructs new scenarios on existing instructions, continuously generating more challenging training samples.

C. Curriculum Learning

Curriculum Learning, first proposed by Bengio et al. [16], is rooted in the "shallow-to-deep" learning paradigm from human education: by strategically organizing the presentation order of training samples—from simple to complex—one can improve training outcomes. The approach has been shown to help models find better local minima, accelerate convergence, and has been applied successfully in many domains [16].

In natural language processing, curriculum learning has been applied to tasks such as machine translation and text classification. In code-related tasks, curriculum learning has also shown promise: some work has sorted training data by bug complexity in program-repair tasks [17] with positive results.

However, applications of curriculum learning in the code-generation fine-tuning setting remain scarce. A key challenge is defining the "difficulty" of code-generation instructions—unlike sentence length in machine translation, the complexity of a code instruction involves multiple dimensions such as logical depth, constraint count, and algorithmic difficulty. This paper uses the instruction-complexity score produced during data filtering as the difficulty measure for curriculum learning, naturally linking the quality-filtering and curriculum-organization stages, and proposes a complexity-graduated curriculum scheme tailored to code generation.

D. Evaluation of Code Generation

Evaluation metrics for code generation have similarly evolved from vague to precise. Early studies borrowed the BLEU [11] score or Exact Match from NLP, but these methods were quickly shown to be ill-suited for code evaluation, as they penalize functionally correct programs that differ in implementation details (variable names, style, etc.).

To address this, the pass@k metric was introduced in the Codex paper [2] and quickly became the gold standard for the field. pass@k evaluates functional correctness by generating k candidate programs and checking whether at least one passes all predefined unit tests. It directly reflects the practical usability of generated code. HumanEval [2], released alongside pass@k, is a benchmark of 164 handcrafted programming problems and has become the authoritative standard for measuring functional correctness.

Beyond functionality, code quality is another important evaluation dimension. CodeBLEU [12] is a complementary execution-free metric that assesses syntactic and semantic quality by comparing the abstract syntax tree (AST) and data-flow graph of generated code against a reference.

III. Methodology

A. Base Model Architecture

We choose Qwen3-0.6B as the base model for this study. It is a decoder-only Transformer [14] with roughly 0.6 B parameters (of which ~0.44 B are non-embedding) arranged in 28 Transformer layers. Its compact scale delivers strong performance, making it an ideal choice for efficient fine-tuning experiments in resource-constrained settings. The model was pre-trained on ~36 T tokens of multilingual text and code,

covering 119 languages, providing a solid foundation of linguistic understanding and code syntax for downstream fine-tuning. Key architectural features include: (1) Grouped-Query Attention (GQA) with an asymmetric configuration of 16 query heads and 8 key-value heads, which preserves attention expressiveness while significantly reducing inference-time memory and compute; (2) the SwiGLU activation function, which provides richer gradient information than ReLU and enhances the feed-forward network's expressiveness; and (3) Rotary Position Embedding combined with RMSNorm pre-normalization, which improves long-sequence modeling and training stability. Additionally, the model introduces QK-Norm in the attention layers and removes the QKV bias, further stabilizing training.

B. Problem Formulation

Our goal is to train a language model M that, given a natural-language instruction I (the prompt), produces functionally correct and high-quality code C. This can be formalized as learning a conditional distribution $P(C \mid I)$. At inference time, given a new instruction I_{new}, the model generates the optimal code segment $\hat{C}$ via a decoding strategy (e.g., beam search or nucleus sampling):

$$\hat{C} = \arg\max_{C} P(C \mid I_{new}). \quad (1)$$

We use supervised fine-tuning (SFT) to optimize a pre-trained language model for better adaptation to code generation. SFT updates the model's parameters θ using a high-quality dataset of (I, C) pairs, $D = \{(I_1, C_1), (I_2, C_2), \dots, (I_N, C_N)\}$.

Concretely, we concatenate the instruction I and code C into a single sequence. The model is trained to predict the next token in this sequence. For a target code consisting of a token sequence $C = (c_1, c_2, \dots, c_T)$, the SFT objective minimizes the negative log-likelihood of the sequence conditioned on I.

This follows the standard causal language-modeling objective. For a single training sample (I, C), the loss $\mathcal{L}_{\mathcal{SFT}}$ is:

$$\mathcal{L}_{\mathcal{SFT}}(\theta) = -\sum_{t=1}^{T} \log P(c_t \mid I, c_{<t}; \theta), \quad (2)$$

where θ denotes the trainable parameters, c_t is the t-th token of the target code, and $c_{<t}$ denotes all tokens preceding c_t in the sequence, i.e., $(c_1, c_2, \dots, c_{t-1})$. $P(c_t \mid I, c_{<t}; \theta)$ is the model's predicted probability for token c_t, typically obtained from the final-layer output via softmax. Over the full dataset D, we minimize the expected loss:

$$J(\theta) = E_{(I,C)\sim D}[\mathcal{L}_{\mathcal{SFT}}(\theta)]. \quad (3)$$

Using a gradient-based optimizer (such as AdamW), the model gradually learns to generate code matching the target distribution given an instruction.

C. Parameter-Efficient Fine-Tuning

Although Qwen3-0.6B is already compact, full fine-tuning of its 0.6 B parameters still incurs considerable memory and compute costs. We therefore adopt Low-Rank Adaptation (LoRA) [13]. The core idea of LoRA is to freeze the pre-trained weights W_0 during fine-tuning and inject two trainable low-rank matrices A and B into specific layers (typically the query and value projections in the self-attention module), where $A \in R^{d\times r}$, $B \in R^{r\times k}$, and $r \ll \min(d, k)$. The forward pass becomes:

$$h = W_0 x + BAx. \quad (4)$$

During training only A and B are updated, reducing the trainable parameter count from hundreds of millions to a few million and substantially lowering memory and time costs.

D. Complexity-Aware Multi-Dimensional Data Quality Filtering

While Evol-Instruct can automatically generate highly complex training data, the iterative generation process inevitably introduces noisy samples. We therefore design a code-specific multi-dimensional quality-evaluation and filtering pipeline that systematically assesses the raw dataset D_{raw} (codefuse-ai/Evol-instruction-66k) along three complementary dimensions.

Dimension 1: Instruction Complexity Score (ICS). We evaluate the intrinsic complexity of each instruction to ensure that the filtered dataset remains sufficiently challenging. We construct the complexity score from the following heuristic features:

$$\mathrm{S_{ICS}(I)} = \mathrm{w_1 f_{constraint}(I) + w_2 f_{depth}(I) + w_3 f_{step}(I)}, \quad (5)$$

where $f_{constraint}(I)$ counts the number of explicit constraints in the instruction (triggered by keywords such as "must," "must not," "simultaneously satisfy"), $f_{depth}(I)$ measures the logical-nesting depth required by the instruction (estimated by analyzing conditional nesting patterns), and $f_{step}(I)$ counts the number of independent sub-tasks. w_1, w_2, w_3 are normalized weights. This score filters out samples that have degenerated to simple instructions during the evolution process, retaining only truly complex training instances.

Dimension 2: Executability Verification. The distinguishing property of code-generation tasks is that outputs are objectively verifiable. We automatically execute the code segment C of each sample in a sandbox environment and detect whether it produces syntax errors (SyntaxError) or runtime exceptions (NameError, TypeError, etc.). The executability indicator is defined as:

$$\mathbb{1}_{exec}(C) = \begin{cases} 1, \textit{the code executes without exceptions}, \\ 0, \textit{the code has syntax or runtime errors}. \end{cases} \quad (6)$$

Non-executable code samples not only fail to provide positive supervision but may actively lead the model to learn incorrect code patterns, and are therefore filtered out.

Dimension 3: Instruction-Code Alignment Score (ICAS). Even when code is executable, it may still fail to match the instruction's semantics—i.e., "off topic." We use a stronger teacher model (Qwen-72B-Chat) to assess the semantic alignment between instruction and code. Specifically, we feed the pair (I, C) into the teacher model with a carefully designed prompt asking it to rate the pair from 1 to 5 along three aspects—functional coverage, edge-case handling, and code style—and take the mean, normalized to $S_{ICAS} \in [0,1]$.

Combined filtering. The three dimensions are combined into a final sample-quality score via a weighted sum:

$$\mathrm{S_{total}}(\mathrm{I},\mathrm{C}) = \lambda_1 \widehat{\mathrm{S_{ICS}}}(\mathrm{I}) + \lambda_2 \mathbb{1}_{\mathrm{exec}}(\mathrm{C}) + \lambda_3 \mathrm{S_{ICAS}}(\mathrm{I},\mathrm{C}), \quad (7)$$

where $\widehat{S_{ICS}}$ is the ICS normalized to $[0,1]$, and $\lambda_1, \lambda_2, \lambda_3$ are dimension weights with $\sum_i \lambda_i = 1$. In our experiments we set $\lambda_1 = 0.3, \lambda_2 = 0.3, \lambda_3 = 0.4$, giving slightly higher weight to alignment because "off topic" samples are the most harmful to learning. With a quality threshold τ, the filtered dataset is:

$$\mathrm{D_{filtered}} = \{(\mathrm{I},\mathrm{C}) \in \mathrm{D_{raw}} \mid \mathrm{S_{total}}(\mathrm{I},\mathrm{C}) > \tau\}. \quad (8)$$

Through this pipeline, starting from a 10k sample subset of Evol-instruction-66k we obtain 5,247 high-quality samples (filter rate ~47.5%). The majority of removed samples are non-executable (the raw data has only a 50.9% executability rate), confirming the necessity of quality filtering for evol-instruct code data.

E. Complexity-Graduated Curriculum Learning Strategy

Traditional SFT randomly shuffles all samples and trains them uniformly, ignoring the difficulty gradient across samples. Inspired by curriculum learning theory [16], we propose a complexity-graduated training strategy that uses the instruction-complexity score S_{ICS} obtained in Section III.D as a natural difficulty measure and organizes training as a progressive easy-to-hard process.

Difficulty bucketing. Based on the $\widehat{S_{ICS}}$ distribution, we partition the filtered dataset $D_{filtered}$ at tertile cutoffs into three difficulty levels:

$$\mathrm{D_{easy}} = \{(\mathrm{I},\mathrm{C}) \in \mathrm{D_{filtered}} \mid \widehat{\mathrm{S_{ICS}}}(\mathrm{I}) \le \mathrm{q_{33}}\}, \quad (9)$$

$$\mathrm{D_{medium}} = \{(\mathrm{I},\mathrm{C}) \in \mathrm{D_{filtered}} \mid \mathrm{q_{33}} < \widehat{\mathrm{S_{ICS}}}(\mathrm{I}) \le \mathrm{q_{66}}\}, \quad (10)$$

$$\mathrm{D_{hard}} = \{(\mathrm{I},\mathrm{C}) \in \mathrm{D_{filtered}} \mid \widehat{\mathrm{S_{ICS}}}(\mathrm{I}) > \mathrm{q_{66}}\}, \quad (11)$$

where q_{33} and q_{66} are the 33rd and 66th percentiles of $\widehat{S_{ICS}}$, respectively.

Progressive training. Training is divided into three stages, each using a cumulatively expanding data subset:

- Stage 1 (foundation building): train for 1 epoch on D_{easy} alone, so the model first masters basic code-syntax patterns and simple logical structures.
- Stage 2 (capability expansion): train for 1 epoch on $D_{easy} \cup D_{medium}$, introducing medium-complexity samples so the model learns richer logical compositions and constraint handling.
- Stage 3 (advanced challenges): train for 1 epoch on the full $D_{filtered} = D_{easy} \cup D_{medium} \cup D_{hard}$, exposing the model to the most challenging complex tasks to fully activate its reasoning potential.

Formally, the stage-s objective is:

$$\mathrm{J_s}(\theta) = \mathrm{E}_{(\mathrm{I},\mathrm{C})\sim \mathrm{D_s}}\left[-\sum_{\mathrm{t}=1}^{\mathrm{T}} \log P\left(c_t \mid I, c_{<t}; \theta\right)\right], s \in \{1,2,3\}, \quad (12)$$

with $D_1 = D_{easy}, D_2 = D_{easy} \cup D_{medium}$, and $D_3 = D_{filtered}$.

Key advantages of this strategy are: (1) the syntactic and basic-logic knowledge acquired from low-complexity samples in the early stages provides a solid foundation for learning complex patterns later; (2) the progressive introduction of high-complexity samples avoids the gradient-instability issues that arise when the model is exposed to overly difficult data too early; and (3) simple samples are reused across multiple stages, acting as an implicit form of data augmentation that consolidates fundamental knowledge.

F. Overall Framework

Combining the components above, the CAFCL framework operates as follows: (1) Evol-instruction-66k serves as the raw data source (we subsample 10k for processing in this work); (2) the complexity-aware multi-dimensional data-quality filtering yields the high-quality dataset $D_{filtered}$ (5,247 samples) along with a complexity score for each sample; (3) the complexity scores are used for difficulty bucketing and the curriculum strategy organizes progressive training; and (4) fine-tuning is performed efficiently under LoRA. The framework integrates data filtering and training organization organically—the complexity scores produced during filtering naturally serve as the difficulty axis for curriculum learning—so that the two components work together to achieve the dual optimization of "high-quality data + principled training order."

IV. EXPERIMENTS

Our experiments evaluate the effectiveness of the CAFCL framework and, through training-dynamics analysis, shed light on its working mechanism. We organize the study around two research questions: **RQ1 -** How large an improvement can CAFCL deliver over existing methods? **RQ2** - How do the dynamics during training reflect the mechanisms of data filtering and curriculum learning?

A. Experiment Setup

Evaluation benchmark. We use HumanEval [2] as our evaluation benchmark. It contains 164 carefully handcrafted Python programming problems, each equipped with a full unit-test suite, and is the gold standard for measuring the functional correctness of code-generation models.

Functional correctness (pass@k): the core of our evaluation. In theory, if the probability of any single generated sample passing all unit tests is p, then the probability that at least one of k generated samples passes—denoted pass@k—is:

$$\text{pass@}k = 1 - (1-p)^k. \quad (13)$$

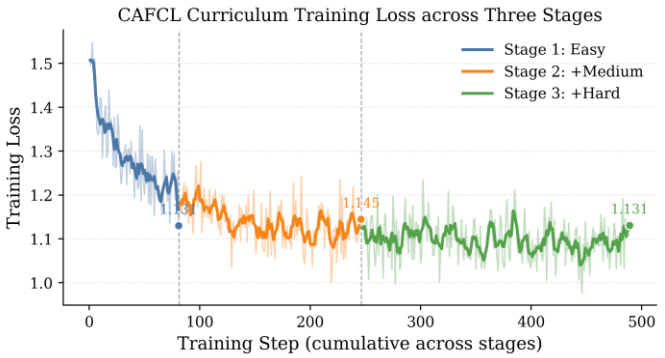


Figure 1. Training loss curve across the three stages of CAFCL. The x-axis is the cumulative step counter across stages; the y-axis is cross-entropy loss. Vertical dashed lines mark stage transitions.

In practice p is unknown, so we estimate it via Monte Carlo. For each problem we independently sample n solutions and count the number c that pass all unit tests; pass@k is then computed by the following unbiased estimator:

$$\text{pass@k} \approx 1 - \frac{\binom{n-c}{k}}{\binom{n}{k}}, \tag{14}$$

where $\binom{n}{k}$ is the binomial coefficient, i.e., the number of ways to choose k items from n.

B. Implementation Details

Model and fine-tuning configuration. We load the model from the official `Qwen/Qwen3-0.6B` checkpoint and fine-tune with LoRA [13], using rank $r = 16$ and scaling factor $\alpha = 32$, applied to the query projection `q_proj` and value projection `v_proj` of all 28 self-attention layers. We use the AdamW optimizer with a cosine-annealing learning-rate schedule, a peak learning rate $\eta_{max} = 2e-5$, and a linear warm-up over the first 10% of steps. The global batch size is 64 (via gradient accumulation), and training is performed on a single NVIDIA RTX 4090 24 GB GPU. All baselines use the same hyperparameters and comparable step budgets to ensure a fair comparison.

Data-filtering configuration. In the instruction-complexity score, the weights for constraint count, logical depth, and sub-task count are $w_1 = 0.4$, $w_2 = 0.35$, $w_3 = 0.25$. The dimension weights of the combined quality score are $\lambda_1 = 0.3, \lambda_2 = 0.3, \lambda_3 = 0.4$. The quality threshold τ was determined by grid search on a held-out set and is set to 0.45. Executability verification runs in an isolated Python 3.10 sandbox with a 10-second timeout.

Curriculum-learning configuration. Difficulty buckets are defined by tertile splits of $\widehat{S_{ICS}}$; the resulting $Easy : Medium : Hard$ ratios are approximately $1772 : 1756 : 1719$ (over the 5,247 samples retained after filtering the 10k subsample of Evol-instruction-66k). All three stages use the same cosine-annealing learning-rate schedule, with one complete schedule independently executed within each stage; each stage trains for 3 epochs. Across stages, the LoRA adapter's weights are inherited, but the optimizer state and LR schedule are reset at the start of each stage.

C. Main Results (RQ1)

TABLE I. PERFORMANCE OF EACH METHOD ON HUMANEVAL (PASS@1, GREEDY DECODING, N=1).

Model	***Training Data***	***pass@1 (%)***
B1: Qwen3-0.6B (Base)	-	28.05
B2: + Evol (Random)	10k full + random order	31.71
B3: + Evol-Filtered (Random)	5.2k filtered + random order	30.49
B4: + CAFCL (Ours)	**5.2k filtered + curriculum (3 stages)**	**32.32**

Basic effectiveness of SFT. All fine-tuned variants (B2–B4) improve pass@1 over the un-fine-tuned base model B1 by 2.4–4.3 percentage points, confirming that SFT on evol-instruct data can further activate the code-generation ability of the already strong Qwen3-0.6B starting point.

Filtering alone has limited impact. B3 uses about half the data of B2, yet its pass@1 is 1.2 percentage points lower than B2 (30.49 vs 31.71). This suggests that, at the current scale, filtering significantly reduces training cost (sample count halved) but does not yield additional performance gains on its own when combined with random-order training, indicating that filtering needs to be paired with an appropriate training-organization strategy.

Curriculum learning delivers a positive but modest additive gain. Introducing the complexity-graduated curriculum on top of B3's filtered data, B4 improves pass@1 by 1.83 points over B3 (32.32 vs 30.49) and by 0.61 points over the random full-data baseline B2. Although the absolute improvement is small, CAFCL achieves better performance using only half as much training data (5.2k vs 10k), demonstrating the value of the filtering-plus-curriculum combination—neither filtering alone nor full-data random training matches their combination.

Overall improvement. CAFCL ultimately reaches pass@1 = 32.32%, improving over the un-fine-tuned base model by 4.27 percentage points and slightly outperforming the random full-data fine-tuning baseline under the same training budget. The combination of a small model (0.6 B) and low-rank LoRA ($rank = 16$) caps the headroom available to CAFCL; we expect CAFCL's advantage to be more pronounced in larger models or with higher-rank fine-tuning configurations.

D. Training-Dynamics Analysis (RQ2)

To gain a deeper understanding of CAFCL's working mechanism from the training-process perspective, we record and analyze the loss trajectories across the three stages of B4's curriculum training.

As shown in Figure 1, the "rebound-then-descend" pattern at stage boundaries. When Stage 2 introduces Medium-difficulty samples and the optimizer/LR are reset at the stage boundary, the training loss rebounds slightly (Stage 1 end 1.131

→ Stage 2 start ~1.17) and then falls and stabilizes around ~1.145. A similar small fluctuation appears when Stage 3 introduces Hard samples, with the final loss converging to 1.131. This "harder data introduced → brief rebound → re-convergence" three-step pattern is consistent with the theoretical expectation of curriculum learning.

Saturation at the match between model capacity and data difficulty. The final losses of the three stages are $1.131/1.145/1.131$—even though Stage 3 contains more and harder samples, the LoRA (rank = 16) parameters are already close to their expressive limit by the end of Stage 1, so further training cannot meaningfully reduce the overall loss. This phenomenon explains why CAFCL's gain over baselines under the current configuration (about 0.6–1.8 pp) is bounded by a capacity bottleneck; in theory, higher-rank LoRA or full-parameter fine-tuning would leave more headroom for the curriculum to exploit.

E. Qualitative Analysis

Successful case. Given an instruction to implement a Python function $parse_complex_data(items)$ that must satisfy four pieces of logic simultaneously—pure numeric strings are converted directly to integers; mixed strings have their digits extracted and concatenated; strings without digits are ignored; the string $'ERROR'$ immediately raises $ValueError$ —the CAFCL-fine-tuned model produces a robust implementation using a try-except block, $isdigit()$ checks, and regular-expression $re.findall$, passing all tests on the first attempt. In contrast, B2 (full-data random training) handles three of the four cases but misses the $'ERROR'$ special branch, and B3 (filtered random training) covers all four but has defects in the regex edge-case handling. This case illustrates the synergy between filtering and curriculum learning in CAFCL—filtering ensures the model learns correct patterns, and curriculum learning ensures it can systematically compose multiple patterns.

Failure case. Given an instruction requiring, without fully loading the data into memory, the median of the merged output of two sorted iterators in $O(\log(m+n))$ complexity, our model's solution was to fully materialize both iterators into lists, concatenate them, sort, and take the middle element. Although correct for small inputs, the solution violates both the time and space complexity constraints. This case exposes the model's reasoning limitations in designing complex algorithms from scratch (such as a binary-search solution for Top-K variants), suggesting that future research must move from instruction following toward genuine algorithm synthesis.

V. Conclusion

This paper presents a supervised fine-tuning (SFT) approach to enhance large language models (LLMs) for Text-to-SQL tasks. The proposed method significantly improves the generation of complex SQL queries and cross-domain adaptability, as demonstrated by experiments on the Spider and CHASE datasets. Results show notable gains in Exact Match (EM) accuracy compared to baseline models, highlighting the effectiveness of fine-tuning LLMs for SQL generation. Future work will focus on further optimizing inference efficiency and enhancing model robustness for real-world applications.

References

[1] Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., ... & Amodei, D. (2020). Language models are few-shot learners. *Advances in neural information processing systems*, *33*, 1877-1901.

[2] Chen, M., Tworek, J., Jun, H., Yuan, Q., Pinto, H. P. D. O., Kaplan, J., ... & Zaremba, W. (2021). Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*.

[3] Li, Y., Choi, D., Chung, J., Kushman, N., Schrittwieser, J., Leblond, R., ... & Vinyals, O. (2022). Competition-level code generation with alphacode. *Science*, *378*(6624), 1092-1097.

[4] Li, R., Allal, L. B., Zi, Y., Muennighoff, N., Kocetkov, D., Mou, C., ... & de Vries, H. (2023). Starcoder: may the source be with you!. *arXiv preprint arXiv:2305.06161*.

[5] Roziere, B., Gehring, J., Gloeckle, F., Sootla, S., Gat, I., Tan, X. E., ... & Synnaeve, G. (2023). Code llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950*.

[6] Yang, A., Li, A., Yang, B., Zhang, B., Hui, B., Zheng, B., ... & Qiu, Z. (2025). Qwen3 technical report. *arXiv preprint arXiv:2505.09388*.

[7] Kocetkov, D., Li, R., Allal, L. B., Li, J., Mou, C., Ferrandis, C. M., ... & de Vries, H. (2022). The stack: 3 tb of permissively licensed source code. *arXiv preprint arXiv:2211.15533*.

[8] Wang, Y., Kordi, Y., Mishra, S., Liu, A., Smith, N. A., Khashabi, D., & Hajishirzi, H. (2023, July). Self-instruct: Aligning language models with self-generated instructions. In *Proceedings of the 61st annual meeting of the association for computational linguistics (volume 1: long papers)* (pp. 13484-13508).

[9] Chaudhary, S. (2023, July). *Code alpaca: An instruction-following llama model for code generation*.

[10] Xu, C., Sun, Q., Zheng, K., Geng, X., Zhao, P., Feng, J., ... & Jiang, D. (2023). Wizardlm: Empowering large language models to follow complex instructions. *arXiv preprint arXiv:2304.12244*.

[11] Papineni, K., Roukos, S., Ward, T., & Zhu, W. J. (2002, July). Bleu: a method for automatic evaluation of machine translation. In *Proceedings of the 40th annual meeting of the Association for Computational Linguistics* (pp. 311-318).

[12] Ren, S., Guo, D., Lu, S., Zhou, L., Liu, S., Tang, D., ... & Ma, S. (2020). Codebleu: a method for automatic evaluation of code synthesis. *arXiv preprint arXiv:2009.10297*.

[13] Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., ... & Chen, W. (2022). Lora: Low-rank adaptation of large language models. *Iclr*, *1*(2), 3.

[14] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., ... & Polosukhin, I. (2017). Attention is all you need. *Advances in neural information processing systems*, *30*.

[15] Zhou, C., Liu, P., Xu, P., Iyer, S., Sun, J., Mao, Y., ... & Levy, O. (2023). Lima: Less is more for alignment. *Advances in Neural Information Processing Systems*, *36*, 55006-55021.

[16] Bengio, Y., Louradour, J., Collobert, R., & Weston, J. (2009, June). Curriculum learning. In *Proceedings of the 26th annual international conference on machine learning* (pp. 41-48).

[17] Ding, Y., Min, M. J., Kaiser, G., & Ray, B. (2024). Cycle: Learning to self-refine the code generation. *Proceedings of the ACM on Programming Languages*, *8*(OOPSLA1), 392-418.